

Những Kiến Thức System Design & Pattern Từ TEL Prober

1 DESIGN PATTERNS

1.1 Message Queue / Message Dispatcher Pattern ★ ★ ★

Định nghĩa: Tất cả communication giữa tasks đi qua 1 central message queue.

Ví dụ trong code:

```
// APPLMainTask (APPLTaskMain.c)
FOREVER {
    pNo = MsgGetPNo(&fId, &tot, &msgNum, &readBuf[0], ...);

    switch(pNo) {
        case 1: APPLLotStartProc(...); break;
        case 2: APPLLoadProc(...); break;
        case 3: APPLMainProc(...); break;
        // ...
    }
    MQSend(tot, sendBuf, ...);
}
```

Lợi ích:

- Centralized control (tất cả xử lý sequential, không race condition)
- Easy debugging (trace 1 message flow)
- Decoupled tasks (không cần biết nhau)
- FIFO processing (orderly execution)

Khi nào dùng:

- Multi-task systems
- Embedded systems, real-time systems
- Networked services (web servers, message brokers)

1.2 State Machine Pattern ★ ★ ★

Định nghĩa: System có nhiều states, transitions hạn chế giữa states.

Ví dụ:

```
States: IDLE → LOAD → ALIGNMENT → TEST → UNLOAD → LOT_END → IDLE
```

Transitions:

- IDLE → LOAD (when PP000 received)
- LOAD → ALIGNMENT (when wafer loaded)
- ALIGNMENT → TEST (when aligned)
- TEST → UNLOAD (when test done)
- ... etc

Code trace:

- [aApplMainSeq.c](#) - Sequence logic
- ApplSeqF flags - State tracking
- Process state checks before actions

Lợi ích:

- Invalid state transitions prevented
- Clear state flow
- Easy to understand & maintain

1.3 Buffer/Queue Pattern ★ ★

Định nghĩa: Data stored in buffer, flushed periodically (not immediately).

Ví dụ: Event logging

```
// File: NewTestingEventLog (RAM buffer)
// Accumulate events during measurement
// Flush to disk at:
// - WAFER_END (new feature we implemented)
// - LOT_END (original)

apSaveNewTestingEventLog(NULL); // Trigger flush
```

Lợi ích:

- Performance (batch writes vs individual writes)
- Atomicity (all events written together)
- Reduces I/O operations

1.4 Handler/Callback Pattern ★ ★

Định nghĩa: Specific function handles specific message type.

Ví dụ:

```
Message Type → Handler Function
PL000 → APPLLotStartProc()
```

```
PP000 → APPLLoadProc()
PP001 → APPLMainProc()
PP002 → RPSuspend()
```

Benefit:

- Easy to add new message types (add new case)
- Each handler focused on one task
- Reusable handler pattern

1.5 Sequence Pattern ★ ★ ★

Định nghĩa: Các bước thực hiện theo sequence (LOT_START → LOAD → ALIGN → TEST → ...).

Ví dụ:

```
BEFORE_TEST sequence:
1. Mark wafer
2. Heat (preheat)
3. Contact (probe touch)
4. Polish (if needed)
```

```
Each step:
- Has preconditions
- Has sub-steps
- Has error handling
- Has success criteria
```

Code: [aBeforeTest.c](#), [aAlignProc.c](#)

1.6 Adapter/Wrapper Pattern ★

Định nghĩa: Translate between different interfaces/protocols.

Ví dụ:

- GEM/HSMS Protocol (PN300 interface) → Internal message format
- GPIB commands → Internal commands
- FTP file format → Internal data structures

Code: [GEM/](#), [HSMS/](#)

[2] SYSTEM DESIGN CONCEPTS**2.1 Real-Time Operating System (RTOS)** ★ ★ ★

Khái niệm: OS quản lý multiple concurrent tasks, priority-based scheduling.

TEL Prober uses: VxWorks (industry standard RTOS)

Concepts:

- **Task:** Independent thread of execution
- **Priority:** Higher priority task preempts lower priority
- **Semaphore/Mutex:** Synchronization primitive
- **Message Queue:** IPC (Inter-Process Communication)
- **Real-time:** Predictable, bounded response time

Code examples:

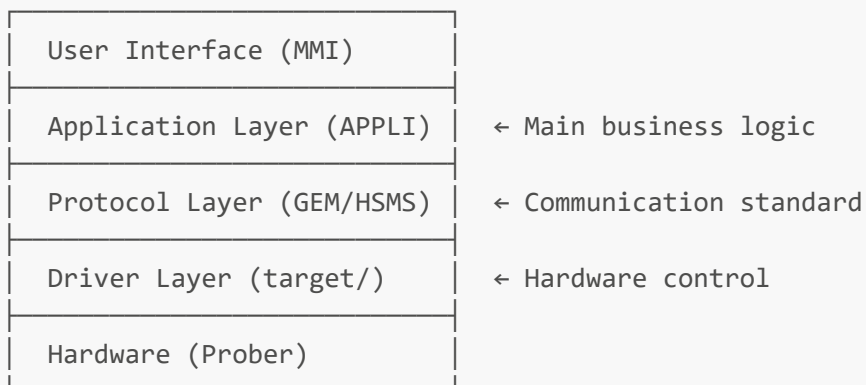
```
// Task creation
ApplTask = TyTaskSpawn(APPLI_TASK, APPL_TASK_PRI, ...);

// Message passing
MQSend(ApplTask, "PP001", WAIT_FOREVER, 0);
pNo = MsgGetPNo(&fId, &tot, ...);

// Synchronization
semGive(semaphore);
semTake(semaphore, WAIT_FOREVER);
```

2.2 Layered Architecture ★ ★ ★

Struktur:



Benefits:

- Separation of concerns
- Each layer has responsibility
- Easy to test/debug individual layers
- Easy to change implementation (e.g., swap protocol)

2.3 Event-Driven Architecture ★ ★

Concept: System responds to events (messages).

Events in TEL Prober:

- Wafer loaded → LOAD event
- Alignment complete → ALIGN event
- Test done → TEST_DONE event
- Lot finished → LOT_END event

Benefits:

- Loose coupling (components don't need to know each other)
 - Reactive (responds to events, not polling)
 - Scalable (easy to add event handlers)
-

2.4 Task Synchronization & Mutual Exclusion ★ ★ ★

Problem: Multiple tasks accessing shared resources → race conditions.

Solutions in TEL Prober:

- **Message Queue:** One task processes at a time (serialized)
 - **Semaphores:** Control access to shared resources
 - **Flags:** State flags (ApplSeqF, WafTestSeqF, etc)
 - **Atomic Operations:** Read/write single variables atomically
-

2.5 State Machine with Flags ★ ★

Example:

```
// File: APPL_main.c
extern uns1 ApplSeqF;           // Application state flags
extern uns1 WafTestSeqF;       // Wafer test state flags
extern uns1 ComMasterF;        // Master comm flags
extern uns1 ComGpibF;          // GPIB comm flags

// Usage in code:
if(ApplSeqF & APLSEQ_IDLE) // Check if idle
if(WafTestSeqF & REP_PHASE) // Check if reprobe phase
```

Benefits:

- Compact state representation (multiple flags in one variable)
 - Fast bit operations
 - Memory efficient
-

3 SOFTWARE ENGINEERING BEST PRACTICES

3.1 Modular Design / Separation of Concerns ★ ★ ★

Principle: Each module has one responsibility.

Modules in TEL Prober:

Module	Responsibility
aLotStart.c	LOT_START logic
aAlignProc.c	Alignment logic
aWafTest.c	Test logic
aLoadProcNew.c	Wafer loading
aBeforeTest.c	Pre-test prep
aRecvResult.c	Result handling
GMSDaDataDiskWrite.c	Data persistence

Benefits:

- Easy to locate code for specific functionality
- Easy to test individual modules
- Easy to reuse modules
- Easy to maintain & modify

3.2 Error Handling & Graceful Degradation ★ ★

Patterns:

- Return status codes (OK, ERROR, NON_MSG, etc)
- Check conditions before action
- Retry logic (e.g., alignment retry count)
- Fallback behaviors

Example:

```
// Alignment with retry
for(retry = 0; retry < AP60_10; retry++) {
    stat = Alignment();
    if(stat == OK) break;
    // Preheat retry if configured
    if(AP60_11) usleep(AP60_11 * 1000000);
}
```

3.3 Configuration Management ★ ★

Types of config in TEL Prober:

- **Setup Files** (*.alf, *.wff, *.pbf): Wafer/lot/probe parameters
- **Recipe Files**: Test conditions, timing
- **Parameter Tables**: Offset, calibration data
- **Feature Flags**: SPO (Special Option) - enable/disable features

Benefits:

- Flexible without code change
 - Customer-specific customization
 - Easy A/B testing
-

3.4 Logging & Tracing ★ ★

Methods:

- **Event Log**: Application-level events
- **Debug Log**: Detailed trace for debugging
- **FTP Upload**: Send logs to customer for analysis

Files:

- NewTestingEventLog - Key events during measurement
 - Debug logs - Detailed trace
-

3.5 API Design ★ ★

Function Signature Pattern:

```
STATUS functionName(
    int1 fromTaskId,           // Who sent this
    intb *inputBuffer,         // Input data
    int1 *toTaskId,           // Response destination
    intb *outputBuffer         // Output data
);
```

Returns:

- OK (0): Success
- ERROR: Failure
- NON_MSG: No message to send
- SEND_MSG: Send response message

Benefits:

- Consistent interface across all handlers
- Clear input/output

- Supports request-response pattern
-

4 SPECIFIC TECHNICAL KNOWLEDGE

4.1 Protocol Design ★ ★ ★

Protocols used:

- **GEM (Generic Equipment Model)**: Standard for fab equipment communication
- **HSMS (High-Speed Message Service)**: Underlying layer for GEM
- **GPIOB**: General Purpose Interface Bus (instrument control)
- **FTP**: File Transfer Protocol

Learning value:

- How industry standards work
 - Strict protocol compliance
 - Error recovery mechanisms
-

4.2 Hardware Control & Device Drivers ★ ★

Examples:

- **Stage Control**: Move X, Y, Z motors
- **Camera Control**: Capture images for alignment
- **Loader Control**: Move wafer cassettes
- **Probe Control**: Upper/lower probe card

Code: [target/](#), [loader/](#)

4.3 Data Structure Design ★ ★

Examples:

- **Wafer parameter structure** (tyWapPara)
- **Result data structure** (ResultDataW)
- **Test log structure** (NewTestingEventLog)
- **Alignment data** (alignment offset, rotation)

Concepts:

- Pack data efficiently (bit fields, unions)
 - Align data for hardware access
 - Design for serialization (network, disk)
-

4.4 File I/O & Data Persistence ★ ★

Patterns:

- Binary file format (GMS - GEM Manufacturing Specification)
- CSV export for analysis
- Atomic writes (write to temp, then rename)
- Batch operations (buffer data, flush periodically)

Code: [aApFile.c](#), [GMSDataDiskWrite.c](#)

5 ADVANCED CONCEPTS

5.1 Concurrency Without Threads ★ ★

Challenge: How to handle concurrent operations without race conditions?

Solution in TEL Prober:

- Tasks are concurrent (multiple running)
- But message queue serializes requests
- State machines prevent invalid transitions
- Atomic flag operations

Lesson: You can achieve safe concurrency with careful design, even with RTOS.

5.2 Auto-Detection & Self-Configuration ★ ★

Example: FTP auto-detection

1. File written to disk with SendFile flag = TRUE
2. ftpTask continuously monitors disk
3. When flag found → automatically select & transfer file
4. No manual intervention needed

Lesson: Design for automation, not manual configuration.

5.3 Backward Compatibility ★ ★

Challenge: How to maintain 30+ years of revisions?

Solutions:

- Keep old code paths, add new branches (#if/#endif)
- Version tracking in code comments
- Preserve parameter formats

Lesson: Backward compatibility requires discipline & planning.

5.4 Test-Friendly Design ★ ★

Features:

- Simulation mode (simulator/)
- Diagnostic modes (diag/)
- Parameter override capability
- Detailed logging for analysis

Lesson: Design for testability from day one.

 RECOMMENDED LEARNING PATH

Beginner (Start here)

- 1. ☒ Message Queue Pattern
- 2. ☒ Task-based Architecture (RTOS concepts)
- 3. ☒ State Machine Pattern
- 4. ☒ Separation of Concerns

Intermediate

- 5. ☒ Layered Architecture
- 6. ☒ Error Handling strategies
- 7. ☒ Synchronization primitives
- 8. ☒ Protocol design basics

Advanced

- 9. ☒ Real-time systems optimization
- 10. ☒ Embedded system design tradeoffs
- 11. ☒ Data consistency & atomicity
- 12. ☒ System monitoring & diagnostics

 KEY TAKEAWAYS

Lesson	From TEL Prober	Applies To
Centralized Dispatch	APPLMainTask message queue	Any multi-task system
Stage-based Processing	LOT_START → LOAD → ALIGN → TEST → UNLOAD	Any workflow system
Buffer Pattern	Event log accumulation	Performance optimization
Protocol Compliance	GEM/HSMS strict adherence	Network systems
Modular Design	Separate .c files per stage	Large codebases
State Machines	ApplSeqF, WafTestSeqF	Complex workflows

Lesson	From TEL Prober	Applies To
Configuration over Code	Setup files, parameters	Flexible systems
Graceful Error Handling	Retry, fallback, recovery	Robust systems

 SYSTEM DESIGN PRINCIPLES DEMONSTRATED

- 1. **Single Responsibility Principle (SRP)**: Each module does one thing
- 2. **Open/Closed Principle**: Open for extension (new cases in switch), closed for modification
- 3. **Dependency Inversion**: High-level modules depend on abstractions (message queue), not low-level details
- 4. **Interface Segregation**: Clear, minimal interfaces between modules
- 5. **DRY (Don't Repeat Yourself)**: Reusable handlers, patterns

 WHERE TO FIND EXAMPLES IN CODE

Concept	File	Line
Message Dispatcher	APPLTaskMain.c	1234
State Tracking	APPL_main.h	Various
Task Synchronization	msgQ.h, semLib.h	VxWorks APIs
Sequence Logic	aApplMainSeq.c	Throughout
Error Handling	APPLMainProc	Various cases
Data Persistence	GMSDaDataDiskWrite.c	Throughout
Protocol Handling	GEM/, HSMS/	Protocol impl

 FURTHER READING

Topics to explore:

- 1. VxWorks RTOS documentation
- 2. GEM protocol specification
- 3. Real-time systems design
- 4. Embedded systems programming
- 5. Factory automation systems
- 6. Equipment control systems